

CSE 321 STUDY QUESTION EXAMPLES

1. Find the asymptotic solutions of the following recurrence relations by Master Theorem and determine which function grows faster.

$$T_1(n) = 4T_1(n/2) + n \log n + 3n^2 - 5$$

$$T_2(n) = 9T_2(n/3) + n$$

(Solution)

$$T_1(n) = 4T_1(n/2) + n \log n + 3n^2 - 5$$

$$a = 4, b = 2$$

$$f(n) = n \log n + 3n^2 - 5 \in \theta(n^2) \Rightarrow d = 2$$

Since $a = b^d = 2^2 = 4$, by Master Theorem, $T_1(n) \in \theta(n^2 \log n)$.

$$T_2(n) = 9T_2(n/3) + n$$

$$a = 9, b = 3$$

$$f(n) = n \in \theta(n) \Rightarrow d = 1$$

Since $a > b^d = 3^1 = 3$, by Master Theorem, $T_2(n) \in \theta(n^{\log_b a}) = \theta(n^2)$.

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{T_1(n)}{T_2(n)} &= \lim_{n \rightarrow \infty} \frac{n^2 \log n}{n^2} \\ &= \lim_{n \rightarrow \infty} \log n \\ &= \infty \end{aligned}$$

$T_1(n)$ grows faster than $T_2(n)$.

2. Consider an array and a linked list, both with n elements. Answer the following questions for both data structures.

(a) Analyze the worst-case time complexity of the following operations. Explain your answer in detail.

i. Accessing the first element.

ii. Accessing the last element.

iii. Accessing any element in the middle.

iv. Adding a new element at the beginning.

- v. Adding a new element at the end.
 - vi. Adding a new element in the middle.
 - vii. Deleting the first element.
 - viii. Deleting the last element.
 - ix. Deleting any element in the middle.
- (b) Analyze the space requirements.

Solution:

- (a) Analyzing the worst-time complexity.

	Array	Linked List
i	Regardless of the size of the array and the index of the element we are trying to access, the cost is $O(1)$. Because the elements of an array are stored at contiguous memory locations.	In the linked list, the elements are not stored at contiguous memory locations. But we can easily access the first element in constant time ($O(1)$) by using the root.
ii	As stated above, regardless of the index, cost is $O(1)$.	We can go to the last elements in the list step by step. This means that we have to traverse the list, which takes $O(n)$ time.
iii	As stated above, regardless of the index, cost is $O(1)$.	Similar to accessing the last element, we should traverse the list until we find what we are looking for. So, we can say that it takes $O(k)$ time where k is the index of the desired element. But in the average case, this is $O(n)$ as well.

	Array	Linked List
iv	To add an element at the beginning of the array, we should shift all of the elements to right, which takes $O(n)$ time.	We can create a new node and assign it as our new root. This will take constant time ($O(1)$) only.
v	If the array is not full, this operation will cost only constant time. But if the array is full, we should copy this array to a larger one, and copying n elements will take $O(n)$ time.	To add a new element to the end, we should go to the end of the list, which takes $O(n)$ time.
vi	Similar to adding an element at the beginning, we should move the elements on the right side of that element. That costs $O(n - k)$, where k is the index of the element we are adding. In general, this operation will cost $O(n)$ time.	First, we should go to the memory location where we want to add the new element. Since accessing an element (which is not the root) takes linear time, this operation will cost $O(n)$ as well.
vii	If we are deleting the first element we should shift the remaining part of the array to the left. Shifting $n - 1$ elements will cost $O(n)$ time.	If we assign the 2^{nd} node of of the list as the root, and then delete the previous root, it will take constant time ($O(1)$)).
viii	We can easily free the last element, There is no need to shift, because there is no element left on the right side. Therefore, this operation takes constant time ($O(1)$).	To delete the last element, first we should reach the node before the last. We free the last element and assign the next address pointer of the previous one as NULL. This will take $O(n)$ time, since accessing that element costs linear time.
ix	If we are deleting a middle element we should shift the remaining part of the array to the left. Shifting $n - k$ elements will cost $O(n - k)$, where k is the index of the element we are deleting. In general, this operation will cost $O(n)$ time.	First, we should access the previous of the element which we want to delete. Then we can change the address of the next element and finally, free the node. Since accessing a middle element takes linear time, this operation will cost $O(n)$ as well.

- (b) While storing n elements in an array simply requires $O(n)$ space, a linked-list has a need of storing pointers too. Therefore it is $O(2n)$ for a linked-list. Even though $O(2n)$ converges to $O(n)$, we can still say that arrays are more beneficial in terms of space complexity.

3. Construct an algorithm that converts a given binary tree with size n to a binary search tree (BST). Make sure you preserve the structure of the tree, i.e. you should not add or delete a node. Write down the pseudo-code of the algorithm, explain your reasoning, and analyze the best-case, worst-case, and average-case time complexities.

Solution:

```
helperFunction (root, keys)
    if root is NULL then
        return
    end if
    helperFunction (root.left, keys)
    root.data ← next element of keys
    helperFunction (root.right, keys)
    return

convertToBST (root):
    if root is NULL then
        return
    end if
    keys ← keys of the tree
    keys ← sort(keys)
    helperFunction (root, keys)
    return
```

Firstly, we traverse the tree and save the keys in a set, then sort them. Once we have the sorted set of keys, we start to update the data of each node (in this way, we preserve the structure of the tree). Since the size of the tree is n , we will make n operations to change the values. But the sorting operation dominates the time complexity of the algorithm. If we use Merge Sort, which costs $O(n \log n)$ both in the best and the worst case, the algorithm above will take $O(n \log n)$ as well (in all cases).